

VCD93K Tutorial

February 17, 2011

Contents

1	VCD93K Tutorial	1
1.1	Introduction	1
1.2	Device Description	2
1.2.1	DUT pin definitions	2
1.2.2	DUT timing definitions	3
1.3	Running VCD93K	4
1.4	Setup.py file content:	5
1.4.1	Explanation of Commands	6
1.4.2	Examine the results	7
1.5	Using VCD93K with a spreadsheet (Excel / Open Office)	8
1.6	Running VCD93K with a CSV (spread sheet) File	8
1.6.1	Explanation of Commands	12

Chapter 1

VCD93K Tutorial

1.1 Introduction

1. **VCD93K concept**

VCD93K software converts an event-based test to an ATE test program for various ATEs. An event-based test contains the simulation results that are represented in (E)VCD event-based formats. The conversion uses pre-defined timing according to the Device Spec for the signals that should appear in the final output files.

The Inputs to VCD93K are design simulation files (Verilog VCD/EVCD dump files).

The Output is the Test Program files to run on 93K tester.

2. **Tutorial goal**

The purpose of this tutorial is to familiarize you with VCD93K software. After completing this tutorial you will be able to convert VCD files to tester format Verigy 93K .

We have chosen a VCD file for this tutorial because the signal direction is not defined in the VCD file. It needs to be defined manually for the InOut pins. Also, IO control signals for direction control should be defined.

1.2 Device Description

1.2.1 DUT pin definitions

Below is a list of the direction of device signals and IO control signals as defined in the device specification:

Pin	Direction	I/O control
A0	Input	-
A1	Input	-
CLK0	Input	-
CLK1	Input	-
CLK2	Input	-
CS_	Input	-
D0	Inout	Controlled by D_dir
D1	Inout	Controlled by D_dir
D2	Inout	Controlled by D_dir
D3	Inout	Controlled by D_dir
D4	Inout	Controlled by D_dir
D5	Inout	Controlled by D_dir
D6	Inout	Controlled by D_dir
D7	Inout	Controlled by D_dir
GATE0	Input	-
GATE1	Input	-
GATE2	Input	-
OUT0	Output	-
OUT1	Output	-
OUT2	Output	-
RD_	Input	-
WR_	Input	-
D_dir	Input	-

Table 1.1: Device Direction definitions

D_dir is Internal IO control pin for D0-D7 signals

Note:

When D_dir IO control signal value = 1 Inout signal direction is an Input.

When D_dir IO control signal value = 0 Inout signal direction is an Output.

1.2.2 DUT timing definitions

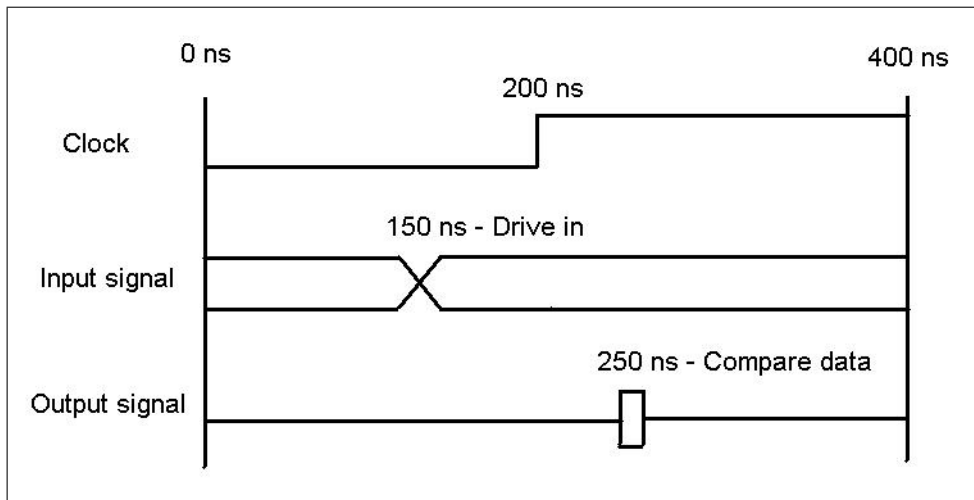


Figure 1.1: Diagram of timing spec for input/output and clock signals

Explanation:

1. The device operates at 2.5Mhz. Therefore we select the tester cycle to be 400ns. We will set the clock to change at 200ns.
2. For input signals we will drive the sample data at 150ns which is 50ns before the posedge of the clock.
3. For output signals we will compare at 250ns which is 50ns after the posedge of the clock.

1.3 Running VCD93K

Change directory to /VCD93K-Tutorials/Basic-Tutorial/

Under this directory you will find the following files:

1. ./Basic-Tutorial/setup.py
2. ./Basic-Tutorial/WR_RD_Mode0.vcd

Run the conversion using the following command:

```
> vcd93k -setup setup.py
```

The generated log for this run is:

```
=====
=   VCD -> STIL   =
=====
.....
.....

Done [stil]

=====
=   STIL -> ATE   =
=====

Start '93000DD' generation ...
.....
.....

Running dd pattern ...
Done

Done.
```

See Tester output file results under ./output/ directory

File Name	File Name	File Name
ai/	data_set/	memt_repair/
shmoo/	vectors/	analog_control/
di_report/	pin_attributes/	state_list/
wafertyp/	applicat/	error_map/
project.tmf	testflow/	waste/
bitmap/	fail_memory/	report/
testflow_vee/	waveform/	bitmap_conf/
format/	routing/	testfunc/
calibration/	levels/	scan/
testprog/	ch_attributes/	memt_llapg/
scan_res/	timing/	configuration/
memt_patterns/	scrambling/	timing_dgrm/

Table 1.2: Table of the files which should be generated under ./output directory for 93K

1.4 Setup.py file content:

In the previous section we ran vcd93k using “-setup setup.py” command line parameters. these command line parameters pass the setup.py configuration file to VCD93K tool. Now lets review the contents of setup.py configuration file:

```
1  input_files = [ 'WR_RD_Mode0.vcd' ]
2
3  cycle_width = '400ns'
4
5  exclude_pins( pinlist = ['D_dir'] )
6
7  pin_attributes(
8      pinlist      = ['CLK?'],
9      type         = 'in',
10     input_waveform = '0ns:D 200ns:U'
11 )
12
13 pin_attributes(
14     pinlist      = ['D?'],
15     type         = 'inout',
16     io_control   = 'in:D_dir=1,out:D_dir=0',
17     input_waveform = '150ns:S',
18     output_waveform = '250ns:S '
19 )
20
21 pin_attributes(
22     pinlist      = ['OUT?'],
23     type         = 'out',
24     output_waveform = '250ns:S'
25 )
26
27 pin_attributes(
28     pinlist      = ['A?','CS_','GATE?','RD_','WR_'],
29     type         = 'in',
30     input_waveform = '150ns:S'
31 )
32
33 target = '93000DD'
```

1.4.1 Explanation of Commands

1. **Line 1:** `input_files = [ 'WR_RD_Mode0.vcd' ]` - The input VCD file

2. **Line 3:** `cycle_width = '400ns'` - The Tester Period

3. **Line 5:** `exclude_pins( pinlist = ['D_dir'] )` - Remove the D_dir pin since this signal is internal IO control pin for D0-D7 signals.

4. **Line 7:** `pin_attributes( pinlist = ['CLK?'], type = 'in', input_waveform = '0ns:D 200ns:U' )`
 Pin_attributes command defines the timing and direction of the pins. You should define the pin attributes command for each pin in your design. In this step we define all the clock pins (CLK0, CLK1 and CLK2) to drive low at 0ns and drive high at 200ns as defined in the device spec.

5. **Line 13:** `pin_attributes( pinlist = ['D?'], type = 'inout', io_control = 'in:D_dir=1,out:D_dir=0', input_waveform = '150ns:S', output_waveform = '250ns:S' )`
 Here we define the pins D0,D1...D7 direction to be inout when the D_dir signal control the direction to be input or output (D_dir=1 for input and D_dir=0 for output). For inout pins it necessary to define both input and output waveforms.

6. **Line 21:**`pin_attributes( pinlist = ['OUT?'], type = 'out', output_waveform = '250ns:S' )`
 We define the pins OUT0, OUT1 and OUT2 to be output and sample the data at 250ns as it was written in the spec.

7. **Line 27:**`pin_attributes( pinlist = ['A?','CS_','GATE?','RD_','WR_'], type = 'in', input_waveform = '150ns:S' )`
 We define all the remaining pins to be input and sample data at 150ns.

8. **Line 33:**`target = '93000DD'` - The output target tester

1.4.2 Examine the results

See Tester output file 'project.tim' under ./output/timing directory

Look at lines 68-89 :

```
SPECS
per_400                [] #

TIMINGSET 1 "WFT"
period=per_400

PINS A1 A0 GATE2 GATE0 GATE1 RD_ CS_ WR_
d1 = 0.375*per_400

PINS CLK0 CLK1 CLK2
d1 = 0.0*per_400
d2 = 0.5*per_400

PINS D6 D7 D4 D5 D2 D3 D0 D1
d1 = 0.375*per_400
r1 = 0.625*per_400

PINS OUT0 OUT1 OUT2
r1 = 0.625*per_400
```

As you can see the timing are at the same points which we defined in the setup file.

1.5 Using VCD93K with a spreadsheet (Excel / Open Office)

In this tutorial we will use a CSV file for editing the pins direction and control.

What is a CSV File?

The Comma Separated Value (CSV) file format is actually a text file format which can be edited as an Excel or Open Office file.

Why use a CSV (spreadsheet) File?

If you have a lot of pins in your design sometimes it is easier to edit all of them in an Excel file and not like in the previous tutorial.

1.6 Running VCD93K with a CSV (spread sheet) File

Change directory to `/VCD93K_Tutorials/Basic_Using_CSV_Tutorial/`

Under this directory you will find the following files:

1. `./Basic_Using_CSV_Tutorial/imported_pins_after_editing.csv`
2. `./Basic_Using_CSV_Tutorial/setup.py`
3. `./Basic_Using_CSV_Tutorial/WR_RD_Mode0.vcd`

Step 1: Fetch pin information from VCD and export it to a CSV file

First we will export all pins from the VCD file into a CSV file format . Then we will open and set up the direction of the pins and define the input/output timing waveform.

For generating the CSV file from WR_RD_Mode0.vcd run the following command:

```
> vcd93k -csv_export_pins imported_pins.csv WR_RD_Mode0.vcd
```

The generated log for this run is:

```
1  =====
2  =   VCD -> STIL   =
3  =====
4
5  Processing Setup...
6  [csv_export_pins] Exporting to 'imported_pins.csv'.
7  Done [setup]
```

Verify that the imported_pins.csv file was created.

Step 2: Update signal direction and timing in the CSV file

Open the CSV file and update all signal directions and timing according to the device spec.

In this tutorial we already did that for you, just compare the two files to see the difference. See 'imported_pins.csv' (the generated CSV file) and the imported_pins_after_editing.csv' (after editing) for reference.

	A	B	C	D	E
1	VS Pin Attributes	1			
2	Pin	Type	I/O Controls	Input Waveform	Output Waveform
3	###HELP###	in/out/inout	in.pin=states,out.pin=states	time:state ... time:state sp=time	time:state ... time:state sp=time
4	D_dir	in			
5	A0	in			
6	A1	in			
7	CLK0	in			
8	CLK1	in			
9	CLK2	in			
10	CS	in			
11	D0	in			
12	D1	in			
13	D2	in			
14	D3	in			
15	D4	in			
16	D5	in			
17	D6	in			
18	D7	in			
19	GATE0	in			
20	GATE1	in			
21	GATE2	in			
22	OUT0	in			
23	OUT1	in			
24	OUT2	in			
25	RD	in			
26	WR	in			

Figure 1.2: CSV file before editing which we generated in the last step

	A	B	C	D	E
1	VS Pin Attributes	1			
2	Pin	Type	I/O Controls	Input Waveform	Output Waveform
3	##HELP##	in/out/inout	in:pin=states	out:pin=states	time:state ... time:state sp=time
4	A0	in		150ns:S	
5	A1	in		150ns:S	
6	CLK0	in		0ns:D 200ns:U	
7	CLK1	in		0ns:D 200ns:U	
8	CLK2	in		0ns:D 200ns:U	
9	CS_	in		150ns:S	
10	D0	inout	in:D_dir=1,outD_dir=0	150ns:S	250ns:S
11	D1	inout	in:D_dir=1,outD_dir=0	150ns:S	250ns:S
12	D2	inout	in:D_dir=1,outD_dir=0	150ns:S	250ns:S
13	D3	inout	in:D_dir=1,outD_dir=0	150ns:S	250ns:S
14	D4	inout	in:D_dir=1,outD_dir=0	150ns:S	250ns:S
15	D5	inout	in:D_dir=1,outD_dir=0	150ns:S	250ns:S
16	D6	inout	in:D_dir=1,outD_dir=0	150ns:S	250ns:S
17	D7	inout	in:D_dir=1,outD_dir=0	150ns:S	250ns:S
18	GATE0	in		150ns:S	
19	GATE1	in		150ns:S	
20	GATE2	in		150ns:S	
21	OUT0	out			250ns:S
22	OUT1	out			250ns:S
23	OUT2	out			250ns:S
24	RD_	in		150ns:S	
25	WR_	in		150ns:S	

Figure 1.3: CSV file after editing

We configure the pin timing and direction as defined in the device spec:

- Cycle period is 400ns
- For clock signals we will drive low at 0ns and drive high at 200ns
- D0-D7 signals will be inout and IO control signal for D0-D7 pins is D_dir pin. For input we sample the data at 150ns and for output we compare at 250ns
- We configure the OUT0-2 to be an output signal and we compare at 250ns
- For all other pins that are input we drive at 150n.
- We have removed D_dir pin from the lists of the pins because it is only a control pin for D0-D7.

Step 3: Run VCD93K setup file

Run the conversion using the following command:

```
> vcd93k -setup setup.py
```

The generated log for this run is:

```
=====
=   VCD -> STIL   =
=====
.....
.....

Done [stil]

=====
=   STIL -> ATE   =
=====

Start '93000DD' generation ...
.....
.....

Running dd pattern ...

Done.
```

[See Tester output file ./output directory was created](#)

Now let's review the contents of the configuration file setup.py:

```
1
2 input_files = [ 'WR_RD_Mode0.vcd' ]
3
4 cycle_width = '400ns'
5
6 csv_import_pins( path= 'imported_pins_after_editing.csv')
7
8 target = '93000DD'
9
```

1.6.1 Explanation of Commands

1. **Line 2:** `input_files = [ 'WR_RD_Mode0.vcd' ]` - The input VCD file
2. **Line 4:** `cycle_width = '400ns'` - The Tester Period
3. **Line 6:** `csv_import_pins( path= 'imported_pins_after_editing.csv')` - Import the CSV file which we edited.
4. **Line 8:** `target = '93000DD'` - The output target tester